

```
In [ ]: %pip install opencv-python
```

```
In [15]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image using OpenCV
image = cv2.imread("image.jpg")

# Check image
if image is None:
    print("Error: image.jpg not found")
else:
    # Convert BGR to RGB for Matplotlib
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(image_rgb)
    plt.title("Image")
    plt.axis("off")

    plt.show()
```

# Talha Shahid

Original Image



Image



## AFFINE → LINEAR → ROTATION

```
In [16]: import cv2
import matplotlib.pyplot as plt

# Load image
image = cv2.imread("image.jpg")

if image is None:
    print("Error: image.jpg not found")
else:
    height, width = image.shape[:2]

    # Center of image
    center = (width // 2, height // 2)

    # Rotation angle
    angle = 45

    # Scale = 1 means no scaling
    scale = 1.0

    # Create rotation matrix
    M = cv2.getRotationMatrix2D(center, angle, scale)

    # Apply rotation
    rotated = cv2.warpAffine(image, M, (width, height))

    # Convert BGR → RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    rotated_rgb = cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB)

    # Display Before and After
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Before: Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(rotated_rgb)
    plt.title("After: Rotation (45°)")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
```

Talha Shahid

Before: Original Image



After: Rotation (45°)



AFFINE → LINEAR → SCALE

Talha Shahid

```
In [17]: import cv2
import matplotlib.pyplot as plt

# Load image
image = cv2.imread("image.jpg")

if image is None:
    print("Error: image.jpg not found")
else:
    # Scaling factors
    scale_x = 3
    scale_y = 3

    # Apply scaling
    scaled = cv2.resize(
        image,
        None,
        fx=scale_x,
        fy=scale_y,
        interpolation=cv2.INTER_LINEAR
    )

    # Convert BGR → RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```



```
scaled_rgb = cv2.cvtColor(scaled, cv2.COLOR_BGR2RGB)

# Display
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Before: Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(scaled_rgb)
plt.title("After: Scaling (3x)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Before: Original Image



After: Scaling (3x)



**AFFINE** → **LINEAR** → **SHEAR**

```
In [18]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
image = cv2.imread("image.jpg")
```



```

if image is None:
    print("Error: image.jpg not found")
else:
    height, width = image.shape[:2]

    # Shearing factor
    kx = 0.5

    # X-shear matrix
    M = np.float32([
        [1, kx, 0],
        [0, 1, 0]
    ])

    # Calculate new width
    new_width = int(width + abs(kx) * height)

    # Apply shear
    sheared = cv2.warpAffine(
        image,
        M,
        (new_width, height)
    )

    # Convert BGR → RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    sheared_rgb = cv2.cvtColor(sheared, cv2.COLOR_BGR2RGB)

    # Display
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Before: Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(sheared_rgb)
    plt.title("After: X-Axis Shearing")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

```

Talha Shahid

Before: Original Image



After: X-Axis Shearing



Rotation + Uniform Scaling + Translation

```
In [19]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
image = cv2.imread("image.jpg")

if image is None:
    print("Error: image.jpg not found")
else:
    height, width = image.shape[:2]

    # Translation values
    tx = 100
    ty = 50

    # Translation matrix
    M = np.float32([
        [1, 0, tx],
        [0, 1, ty]
    ])

    # Apply translation
    translated = cv2.warpAffine(
```



```

    image,
    M,
    (width, height)
)

# Convert BGR → RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
translated_rgb = cv2.cvtColor(translated, cv2.COLOR_BGR2RGB)

# Display
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Before: Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(translated_rgb)
plt.title("After: Translation (100, 50)")
plt.axis("off")

plt.tight_layout()
plt.show()

```

Before: Original Image



After: Translation (100, 50)



## RIGID TRANSFORMATION

```

In [20]: import cv2
import matplotlib.pyplot as plt

# Load image
image = cv2.imread("image.jpg")

if image is None:
    print("Error: image.jpg not found")
else:
    height, width = image.shape[:2]

    # Center
    center = (width // 2, height // 2)

    # Rotation
    angle = 30

    # Scale MUST be 1.0 for rigid transformation
    scale = 1.0

    # Create rotation matrix
    M = cv2.getRotationMatrix2D(
        center,
        angle,
        scale
    )

    # Add translation
    M[0, 2] += 100
    M[1, 2] += 50

    # Apply transformation
    rigid = cv2.warpAffine(
        image,
        M,
        (width, height)
    )

    # Convert BGR → RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    rigid_rgb = cv2.cvtColor(rigid, cv2.COLOR_BGR2RGB)

    # Display
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Before: Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(rigid_rgb)
    plt.title("After: Rigid Transformation")
    plt.axis("off")

```

Talha Shahid



```
plt.tight_layout()
plt.show()
```

Before: Original Image



After: Rigid Transformation



## PROJECTIVE → PERSPECTIVE

```
In [21]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
image = cv2.imread("image.jpg")

if image is None:
    print("Error: image.jpg not found")
else:
    height, width = image.shape[:2]

    # Original corners
    src_points = np.float32([
        [0, 0],
        [width - 1, 0],
        [width - 1, height - 1],
        [0, height - 1]
    ])

    # New perspective positions
    dst_points = np.float32([
```



```

        [100, 50],
        [width - 150, 0],
        [width - 50, height - 80],
        [50, height - 20]
    ])

    # Homography matrix
    M = cv2.getPerspectiveTransform(
        src_points,
        dst_points
    )

    # Apply perspective
    perspective = cv2.warpPerspective(
        image,
        M,
        (width, height)
    )

    # Convert to RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    perspective_rgb = cv2.cvtColor(perspective, cv2.COLOR_BGR2RGB)

    # Display Before / After
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(image_rgb)
    plt.title("Before: Original Image")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(perspective_rgb)
    plt.title("After: Perspective Transformation")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

```

Talha Shahid

Before: Original Image



After: Perspective Transformation



Talha Shahid